Continuous QA Data Environment & Automated Data Quality / Regression Framework for Terminus (PCBINIT-1799)

[PCBDF-2899] Build automated data quality & regression testing framework for Terminus platform

Created: 02/Feb/26 Updated: 12/Mar/26

Status:	To Do
Project:	PCB: Data Foundation
Components:	None
Affects versions:	None
Fix versions:	None
Parent:	Continuous QA Data Environment & Automated Data Quality / Regression Framework for Terminus

Type:	Epic	**Priority:**	4-Low
Reporter:	Chris Chalissery	**Assignee:**	Unassigned
Resolution:	Unresolved	**Votes:**	0
Labels:	None		
Remaining Estimate:	Not Specified		
Time Spent:	Not Specified		
Original estimate:	Not Specified		

Epic Name:	Build automated data quality & regression testing framework for Terminus platform
Parent Link:	PCBINIT-1799Continuous QA Data Environment & Automated Data Quality / Regression Framework for Terminus
Team:	

Description

As part of the story, we want a standardized, automated testing framework for data pipelines and datasets in Terminus.so that we can continuously validate data quality, detect regressions across ~1,000+ DAGs, and ensure trusted data in BigQuery with minimal manual effort.

High-level Requirements

1. **Test Types & Scope**
   - Support:
     - Schema validation (structure, types, partitioning, clustering)
     - Data quality checks (nulls, duplicates, referential integrity, business rules)
     - Distribution/threshold checks (row counts, sums, value ranges, drift)
     - Regression tests when:
       - shared code libraries change
       - pipeline logic changes
       - infrastructure/IaC changes (Terraform/Terragrunt)
   - Work across:
     - BigQuery datasets and tables in Terminus
     - GCS-based staging areas (where applicable)
     - Batch and streaming pipelines (Airflow DAGs, Spark jobs, Dataplex, etc.)
2. **Standardization & Configuration**
   - Define a **standard way to declare tests** (e.g., YAML or config files in repo) aligned with:
     - Terminus Data Platform Strategy: https://pc-technology.atlassian.net/wiki/spaces/PCFENG/pages/8610054409/Terminus+Data+Platform+Strategy
   - Re-use/extend existing **Dataplex quality scan** patterns and metadata standards where possible.
   - Allow per-domain or per-dataset overrides while enforcing a **platform baseline**.
3. **Automation & CI/CD Integration**
   - Integrate with existing CI/CD used for Terminus (Terraform/Terragrunt, DAG deployments):
     - Run relevant tests on PRs / merge to `main/master`.
     - Run scheduled regression suites (e.g. nightly) for critical pipelines.
   - Provide:
     - Automatic impact analysis to select which tests to run when shared code changes (to avoid testing all 1,000 DAGs every time).

        - Clear pass/fail signals back into CI (blocking releases on critical failures).

4. **Platform-Level Monitoring & Reporting**
    - Central dashboard(s) showing:
        - Test status by domain, dataset, pipeline (pass/fail, last run time).
        - Quality metrics (e.g., % of rows failing checks, top recurring rules).
    - Integration with existing observability:
        - Slack alerts (aligned with current Terminus monitoring practices).
        - Grafana / monitoring views if consistent with current approach.
    - Support for:
        - Production PIV-style checks after release.
        - UAT sign-off based on automated test results (aligned to DoD / test strategy on Terminus page: https://pc-technology.atlassian.net/wiki/spaces/OT/pages/6261342209/Terminus+Data+Platform
5. **Security, Compliance & Data Handling**
    - Ensure:
        - Tests respect data masking/tokenization standards (for PII and sensitive data).
        - No raw/un-encrypted PII is written to non-approved locations (including logs, temp tables, or GCS beyond allowed patterns).
    - Use RBAC consistent with existing Terminus security strategy (BigQuery, Dataplex, etc.).
    - Support for **non-production safe datasets** for regression (e.g., sampled/obfuscated data).
6. **Usability for Outcome Teams**
    - Provide:
        - Simple APIs / libraries / templates for outcome teams to onboard new datasets to the framework.
        - Clear documentation and examples for:
            - Adding tests to a new table or DAG.
            - Debugging and triaging failures.
    - Minimize the need for teams to build one-off custom checks.
7. **Support Model & Ownership**
    - Define:
        - Who owns the framework
        - How incidents are handled across Tiers 1–3, aligning with the Terminus Data Platform Suppo
    - Integrate with(TBD):
        - Jira for automated ticket creation on critical recurring failures.

Comments

Comment by Automation for Jira [03/Feb/26]

This is an automated action to sync Epic Name to Summary

Epic Name did not match Summary.
Original Epic Name =

Generated at Fri Mar 27 17:27:18 UTC 2026 by Harshit Patel using Jira 1001.0.0-SNAPSHOT#100291-rev:aa788f054b3aa9723d0aca58424dc5943bb8a5fd.